

Virtual Functions, Polymorphism & Abstract Classes

Early Binding • Late Binding • Overriding • Pure Virtual Functions • Abstract Base Class

- | 1 Recap: Inheritance & Base-class Pointers
- | 2 Early Binding (Compile-time)
- | 3 The Problem it Creates
- | 4 Virtual Functions & Late Binding (Run-time)
- | 5 Function Overriding
- | 6 Pure Virtual Functions
- | 7 Abstract Class / Abstract Base Class
- | 8 Summary & Comparison

Recap: Inheritance & Base-Class Pointers

- 1 A **derived class** inherits data and functions from a **base class**.
- 2 C++ allows a **base class pointer** to point to a **derived class object**.
This is the foundation of runtime polymorphism.
- 3 The big question: when we call a function through that base pointer, **whose function actually runs** — the base class's, or the derived class's?

KEY QUESTION FOR TODAY

Does C++ decide "whose function runs" at **compile time**, or at **run time**? The answer depends on whether the function is **virtual**.

```
class Animal {
public:
    void speak() {
        cout << "Some sound";
    }
};

class Dog : public Animal {
public:
    void speak() {
        cout << "Woof!";
    }
};

// Base pointer, Derived object
Animal* a = new Dog();
a->speak(); // ??? which one runs
```

Early Binding (Static / Compile-time Binding)

- The compiler decides **which function to call** while compiling the program — before it even runs.
- The decision is based on the **type of the pointer/reference**, NOT the actual object it points to.
- This is the **default behavior** in C++ for normal (non-virtual) member functions.
- Also called **Static Binding** because the link between the call and the function is fixed/static.

```
// speak() is NOT virtual here
Animal* a = new Dog();
a->speak();

// Output: "Some sound"
// Compiler sees pointer type = Animal
// so it calls Animal::speak()
// even though the object is a Dog!
```

RULE

With early binding: **Pointer type decides** which function runs — the object's real type is ignored.

Why Early Binding is a Problem

THE REAL-WORLD ISSUE

We often store many different derived objects (Dog, Cat, Cow) using base-class pointers (Animal*), for example in an array or a list. We **expect** each object to speak in its own way — but early binding always calls the base class version.

- Polymorphism promises: "let the object decide its own behavior."
- Early binding breaks that promise — it always asks the **pointer type**, not the object.
- We need a mechanism where the decision is delayed until **runtime**, so the actual object type is checked.
- That mechanism is the **virtual** keyword → next topic.

```
Animal* zoo[3];
zoo[0] = new Dog();
zoo[1] = new Cat();
zoo[2] = new Cow();

for(int i=0; i<3; i++)
    zoo[i]->Speak();

// Expected: Woof, Meow, Moo
// Actual (no virtual):
// "Some sound" 3 times!
```

Virtual Functions

- A **virtual function** is a member function declared in the base class using the **virtual** keyword.
- It tells the compiler: "**Don't decide now — wait and check the actual object at runtime.**"
- When called through a base pointer/reference, C++ now looks at the **real object type**, not the pointer type.
- This behavior is called **Late Binding** (see next slide).

RULE

With virtual functions: **Object type decides** which function runs — this is true runtime polymorphism.

```
class Animal {
public:
    virtual void speak() {
        cout << "Some sound";
    }
};

class Dog : public Animal {
public:
    void speak() {
        cout << "Woof!";
    }
};

Animal* a = new Dog();
a->speak(); // Output: "Woof!"
```

Late Binding (Dynamic / Run-time Binding)

- The function call is resolved **while the program is running**, not while compiling.
- C++ checks the **actual object** pointed/referenced to, and calls its version of the function.
- Made possible internally through a mechanism called the **vtable** (virtual table) — each object with virtual functions keeps a hidden pointer to the correct function.
- Happens **only** when the function is marked **virtual** and it is called via a pointer or reference.

EARLY VS LATE — ONE LINE EACH

Early Binding: decided at compile time, based on pointer type.

Late Binding: decided at run time, based on object type.

REMEMBER

Late binding is **slightly slower** (extra lookup at runtime) but gives real flexibility — this trade-off is the heart of polymorphism.

Function Overriding

- **Overriding** means redefining a base class's **virtual function** in a derived class, with the **exact same signature** (name, parameters, return type).
- It is the mechanism that actually supplies the "derived-class version" that late binding calls at runtime.
- Optional keyword **override** (C++11+) can be added in the derived class — it helps the compiler catch mistakes if the signature doesn't actually match.

DON'T CONFUSE WITH OVERLOADING

Overriding: same function, base & derived class, needs virtual, resolved at runtime.

Overloading: same function name, different parameters, same class, resolved at compile time.

```
class Animal {
public:
    virtual void speak() {
        cout << "Some sound";
    }
};

class Dog : public Animal {
public:
    void speak() override {
        cout << "Woof!";
    }
    // This is OVERRIDING
    // same name, same params
};
```

Pure Virtual Functions

- A **pure virtual function** is a virtual function that has **no body/definition** in the base class.
- Declared by assigning **= 0** at the end of the declaration.
- It only specifies "**what**" a derived class must do — not "**how**." It's a contract.
- Any derived class **must override** it, or that derived class also becomes abstract.

WHY USE IT?

Sometimes the base class truly has **no sensible default** implementation. E.g. a generic "Shape" cannot define **area()** meaningfully — only Circle, Square etc. can.

```
class Shape {
public:
    // pure virtual function
    virtual double area() = 0;
};

class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() override {
        return 3.14 * r * r;
    }
};
```


Abstract Class / Abstract Base Class

- A class that contains **at least one pure virtual function** is called an **Abstract Class**.
- You **cannot create an object** of an abstract class directly.
- It can still have: constructors, normal functions, data members, and even some regular virtual functions with bodies.
- When used purely as a base for inheritance (defining a common interface), it is often called an **Abstract Base Class (ABC)**.
- You **can** still use a pointer/reference of the abstract type — that's exactly how late binding is used in real designs.

```
Shape s; // ERROR!  
// cannot instantiate  
// abstract class  
  
Shape* s = new Circle(5);  
cout << s->area();  
// OK! pointer to abstract  
// class is allowed
```

BIG PICTURE

Abstract classes + pure virtual functions let us design a common **interface** ("every Shape must have an area()") while forcing each derived class to supply its own implementation.

Summary & Comparison

Term	Meaning
Early Binding	Function call resolved at compile time , based on pointer/reference type.
Late Binding	Function call resolved at run time , based on the actual object's type. Requires a virtual function.
Virtual Function	A base class function marked virtual so it can be redefined by derived classes and resolved via late binding.
Overriding	Redefining a base class's virtual function in a derived class with the same signature .
Pure Virtual Function	A virtual function with no body, declared using = 0 . Derived classes must override it.
Abstract Class	A class with at least one pure virtual function. Cannot be instantiated directly, only inherited from.

ONE-LINE MEMORY HOOK

Virtual function → enables late binding → makes overriding meaningful → pure virtual pushes it further → abstract class is the class that "only defines the contract."

END OF SESSION

Questions?

Quick recap: Early Binding → Problem → Virtual Functions & Late Binding → Overriding → Pure Virtual → Abstract Class
